S

P

S

E

Swapcast finds podcasts your size whose listeners like yours, checks that yours like them back, drafts the promo-swap pitch with Qloo evidence, and lists sponsor leads.

I

The median podcast episode gets about 27 downloads in its first week (Buzzsprout, Sep 2026). We kept hearing the same advice for growing a small show: do a promo swap, where two shows promote each other. It is the best free growth tactic, and platforms like Spotify recommend it. But it only works with the right partner, one who is roughly your size, sits in an adjacent lane instead of competing head-on, and whose audience actually likes your show back. Finding that partner is pure guesswork today. A plain LLM just lists the famous shows in your genre, and those shows are too big to say yes and compete with you directly. We wanted a tool that uses a real taste graph to answer the question a podcaster actually has: "Who should I swap with, what do I offer, and how do I ask?"

W

- Finds partners your size. You pick your show (you confirm the exact match, the agent never guesses). Swapcast pulls podcasts whose listeners overlap yours, filtered to a popularity band around your own size so the suggestions are reachable.
- Checks the fit both ways. For each candidate it runs the query in reverse to see where *your* show ranks in *their* listeners' list. A two-way match (strong in both directions) is flagged separately from a one-sided one.
- Drafts the pitch. Each partner gets an editable, copyable promo-swap or guest-swap pitch with the Qloo evidence baked in and a tracking-link reminder so both sides can measure lift.
- Surfaces sponsor leads. It lists brands your listeners already favor, split into reachable vs household names.

T

The agent loop: resolve show â' listener taste profile â' peer shows in your size band (widen the band if too few) â' reverse two-way rank checks on the top candidates â' sponsor brands â' rank by weakest-link and draft pitches.

W

S

-

-
- 3. /v2/insights on urn:entity:podcast with a filter.popularity.min/max band to find peers your size, not just the biggest shows in the genre. The agent widens the band (0.05 â' 0.10 â' 0.20) when peers are thin and logs that decision.
- 4. Reverse insights queries per candidate to compute a two-way rank â" a non-obvious use of the taste graph that checks whether their listeners like you back, not just the other way around.
-

Scoring uses ranks inside a shared size band rather than raw affinity scores, because Qloo normalizes affinity within each query, so ranks compare cleanly while raw scores across queries do not. We sort by weakest = max(fwdRank, revRank ?? 51).

If it would work the same without Qloo, it is the wrong thing. Remove Qloo and there is nothing left: no size band, no two-way check, no cross-domain brands. (Remove the LLM and it still works â" the deterministic pipeline produces every ranking and falls back to template pitches. Only the pitch prose is lost.)

H

- Next.js (App Router) + TypeScript + Tailwind, one repo for UI and server routes, deployed on Vercel.
- Server-only Qloo client (lib/qloo.ts): X-Api-Key header, 8s timeout, two retries with backoff on 429/5xx, a 6-hour in-memory TTL cache keyed by request, and a trace callback that never logs the key.
- Agent pipeline (lib/pipeline.ts): the tool loop with band widening, parallel reverse-rank checks, and a deterministic fallback that runs the same steps if the model is slow or unavailable.
- Scoring (lib/scoring.ts): weakest-link ranking, Jaccard on genre tags to decide same-lane (guest swap) vs adjacent-lane (promo swap), size-match labels.
-
- Guards: every cited ID comes from a Qloo result; numbers shown in the UI come from the server, never from LLM free text. Keys are server-only, .env* is gitignored, and no Qloo response data is committed.

C

- Comparing affinity across queries. Qloo normalizes affinity within each query, so raw scores are not comparable across calls. We switched to rank-based scoring inside a shared popularity band, which is both correct and easier for a user to read.
- Thin peer results for some shows. Small or niche shows returned few peers in a tight band, so the agent widens the band in steps and records why in the trace.
- Latency inside a serverless timeout. Reverse checks multiply the number of calls, so we run them in parallel, cap take, cache aggressively, and stream partial progress to stay well under the limit.
- Keeping evidence honest. We separated "Qloo says" (ranks, tags, brands) from "Swapcast suggests" (format, pitch) throughout the UI so nothing reads as a claim about an individual listener.

W

- Two-way (reverse) querying turns a recommendation API into a genuine matching tool â" it is the single most useful thing we did with Qloo.
- Popularity bands are the difference between "name-drop the biggest show in the genre" and "find a partner who might actually say yes."
- A deterministic fallback is not just a safety net; it forced a clean separation between facts (from Qloo) and prose (from the LLM), which made the whole product more trustworthy.

W

-
- A "Plain LLM vs Swapcast" panel showing judges the gap between a model's guess and evidence-backed matches.
-
- A white-label "Grow" tab for podcast hosting platforms, and later an opt-in swap-and-sponsor marketplace.

L

-
-
-
-

B

n

L

-
-

S

-
-
-

S

-
-
-

-
-
-
-
-

S

F
R

T

P
"The median podcast episode gets 27 downloads in its first week. The single best free way to grow is a promo swap - two shows promote each other. But most swaps fail, because people pick the wrong partner."

I
"A good partner is three things at once: your size, in an adjacent lane so you don't cannibalize each other, and their audience likes you back - not just you liking them. A plain language model can't check any of that. It just name-drops the biggest show in your genre. Only a taste graph can. That's Qloo."

L
"Here's Swapcast. I pick my show - and I confirm the match, the agent never guesses for me. Watch the trace: it pulls my listeners' tastes, finds shows in my size band, then runs each candidate in reverse to check the fit both ways.

[point at a two-way match] This one: Qloo ranks them number 2 for my listeners, and ranks me number 4 for theirs. That's a real mutual fit.

[open the pitch] It drafts the outreach with that evidence baked in, plus a tracking link so we both measure the lift. And over here - brands my listeners already favor, as sponsor leads."

B
"Jason, you run podcasts, so you know this pain. The buyers are the two-plus million podcasters and the hosting platforms that want to keep them growing. Freemium for creators, a Pro tier for tracking and a partner CRM, and a white-label 'Grow' tab for hosting platforms. US podcast ads were 2.9 billion dollars last year. Swapcast turns the taste graph into the one thing a small creator can't get anywhere else: the right partner, and the pitch to win them. Thank you."

D

- Pre-load the demo. Have the show already typed and the result cached (run it once right before) so it

appears in seconds. Never wait on a cold API call on stage.

-
- Separate Qloo from you. Say "Qloo says #2" vs "Swapcast suggests a promo swap." Mike Diolosa (Qloo CTO) is judging the depth of Qloo use.
-
-
- Fallback if the demo stalls: "While that loads - the key move is the reverse check, their listeners liking me back, which is what makes a swap actually convert." Then continue.

O

"Swapcast uses Qloo's taste graph to find podcasts your size whose listeners like yours - and checks that yours like them back - then drafts the promo-swap pitch with the evidence. It turns podcast growth from a guess into a match."

D

Run these from d:\qloo hackathon. You need Node 18.17+ and a GitHub account. The sandbox blocks npm/node for the agent, so run these yourself in a normal terminal.

1

`

n

n

`

l

2

`

n

open http://localhost:3000, click the "This Week in Startups" chip

`

l

3

`
g
If you have an LLM key set, check it too. Both must return nothing.

`

4

`
g
g
g
g
Create an EMPTY public repo on github.com named "swapcast", then:

g
g
`
C

5

O
-
-
-
-
-
-

O
`
n
v
v
v
v
`

6

-
-
-
-
-

7

Use DEVPOST_SUBMISSION.md. Fill the two links (demo URL + repo URL), upload 3 screenshots, add team members, and click Submit (not Save).

I

The app is designed to run with zero config in demo mode. As long as npm run build passes and the Vercel URL loads, you have a qualifying live demo even before the Qloo key is wired in. Add the key via env vars and redeploy (Vercel does this in ~1 min) any time before the deadline.